

Lista de Exercícios n° 8

Capítulo 3 - Linguagem de Programação Haskell

1. Escreva uma função Haskell para calcular o resto inteiro da divisão de dois inteiros positivos, utilizando apenas subtrações sucessivas.
2. Escreva uma função Haskell para calcular o quociente inteiro da divisão de dois inteiros positivos, utilizando apenas subtrações sucessivas.
3. Para as duas funções anteriores, apresente as respectivas definições recursivas em notação matemática, identificando o caso base e o caso recursivo.
4. Escreva uma função Haskell para calcular o quociente inteiro da divisão de dois inteiros, utilizando recursão de cauda.
5. Escreva uma função Haskell para calcular a soma dos números inteiros de 1 até n , utilizando uma definição recursiva.
6. Apresente uma definição recursiva, em notação matemática, para a função do exercício anterior. Em seguida, mostre as chamadas realizadas para o cálculo de soma 5.
7. Considere a sequência definida por

$$T_n = \begin{cases} 1, & \text{se } n = 1, \\ T_{n-1} + 2n - 1, & \text{se } n > 1. \end{cases}$$

Escreva uma função Haskell para calcular T_n . Calcule, utilizando a função, os cinco primeiros termos da sequência e formule uma conjectura para uma expressão não recursiva de T_n .

8. O algoritmo de Euclides para o cálculo do máximo divisor comum pode ser descrito pela recorrência

$$\text{mdc}(a, b) = \begin{cases} a, & \text{se } b = 0, \\ \text{mdc}(b, a \bmod b), & \text{se } b > 0. \end{cases}$$

Implemente essa definição em Haskell e apresente a sequência de chamadas realizadas para calcular `mdc 48 18`.

9. Apresente uma função Haskell que execute um processamento semelhante ao descrito pela função C a seguir:

```
void soma(int N, int *R) {
    int i;
    *R = 0;
    for (i = N; i > 0; i--)
        if (i % 2 == 1)
            *R += i;
}
```

Explique qual valor é calculado pela função e apresente uma definição recursiva, em notação matemática, para esse mesmo cálculo.